CSC 251 Lecture 08: Exception Handling and File I/O

Exception Handling

Any programming error falls into one of three general categories.

Syntax Errors
A syntax error occurs when the interpreter reads code that does not follow the rules of the language. This can occur when you forget a colon, your indentation is incorrect, or you have a typo. For example,

`prit("Hi")` instead of `print("Hi")`

Logic Errors
This type of error occurs when the program runs, but the output is incorrect. For example:

`a + b * c` instead of `(a + b) * c`

Run-Time Errors (**Exceptions**)
These errors occur when your syntax is fine, but the program crashes while running due to an unforeseen problem. When an exception is thrown an object of the exception class is created. For example:

> Your code divides one number by another, but the user enters a zero for the denominator. This will throw a `ZeroDivisionError`.
>
> Your code asks the user for an integer (and it tries to convert the input to an int), but the user types in "muffins!". The program crashes. This will throw a `ValueError`.
>
> Your program tries to open a file for reading, but the file does not exist, or cannot be found in the specified location. The program can't continue. This will throw an `IOError`.

Example 1a: Dividing by Zero

```
def main():
    num1 = int(input('Enter a number: '))
    num2 = int(input('Enter another number: '))

    quotient = divide(num1, num2)
    print(f'{num1} divided by {num2} is {quotient}.')

def divide(num1, num2):
    return num1 / num2

if "__name__" == "__main__":
    main()
```

What can go wrong? ← inappropriate user input that causes a problem here → division by zero

If a 0 is entered for num2, we will get a traceback error, something like:

```
Enter a number: 9
Enter another number: 0
Traceback (most recent call last):
  File "division.py", line 14, in <module>
    main()
  File "division.py", line 9, in main
    result = num1 / num2
ZeroDivisionError: division by zero
```

Execution will halt when the exception is thrown!

Solutions:

1) Input validation

2) Exception handling code in a try/except block

```
try:
    statement
    statement

except exceptionName(optional):
    statement
    statement
```

Which includes a "try suite" that might raise the exception and an "exception clause" that includes a block of statements known as the "handler."

When a try suite is executed:

- If a statement in the try suite raises an exception specified by the *exceptionName* in the except clause, then the handler following that except clause executes. Execution then resumes with the statement AFTER the try/except block.
- If an exception is raised in the try suite that is NOT specified by an *exceptionName*, the program halts.
- If no exceptions are raised in the try suite, the code in the except clause and handlers are skipped and execution resumes after the try/exception block.

Example 1b: Back to our division by zero code:

```
try:
    num1 = int(input('Enter a number: '))
    num2 = int(input('Enter another number: '))

    quotient = num1/num2
    print(f'{num1} divided by {num2} is {quotient}.')

except ZeroDivisionError:
    print('Error: Cannot divide by zero!')
```

all included

Custom message

If 0 is not entered, the program works properly:

```
Enter a number: 3
Enter another number: 2
1.5
```

However, if 0 is entered, the exception is caught and the program does not crash:

```
Enter a number: 9
Enter another number: 0
Error: Cannot divide by zero!
```

← Ouch!

Handling Multiple Exceptions

What if the user enters the following:

```
Enter a number: 9
Enter another number: zero
Traceback (most recent call last):
  File "division1exception.py", line 26, in <module>
    main()
  File "division1exception.py", line 6, in main
    num2 = int(input('Enter another number: '))
ValueError: invalid literal for int() with base 10: 'zero'
```

Example 1c: Let's include an exception clause for this issue:

```
try:
    num1 = int(input('Enter a number: '))
    num2 = int(input('Enter another number: '))

    quotient = num1/num2

    print(result)

except ZeroDivisionError:
    print('Error: Cannot divide by zero!')

except ValueError:
    print('You must enter a numerical value!')
```

Now either type of exception can be handled:

```
Enter a number: 9
Enter another number: zero
You must enter a numerical value!
```

You can add an except clause without an exception name to catch all other exceptions:

```
except:
    print('Error! Something went wrong!')
```

OR you can replace both exception clauses with the single except clause above. What's the downside?

doesn't indicate what went wrong!

To display an exception's default error message:

```
except ValueError as err:
    print(err)
    print('You must enter a numerical value!')
```

(See `division_exceptions.py`)

Combining Exception Handling with Input Validation

Example 2a: If an error is thrown, you may want to prompt the user to enter their value again until it's correct:

```
b = True
while b:
    try:
        n = int(input("Enter an integer: "))
        b = False
    except ValueError:
      print("Invalid integer. Try again.")
```

or `while True:`

`break`

← only executed if no exception thrown

Example 2b: For input validation with more restrictions, here an integer between 0 and 100 (exclusive):

```
b = True
n = -1
while b:
    try:
      while n < 0 or n > 100:
          n = int(input("Enter an integer between 0 and 100: "))
          if n < 0 or n > 100:
             print(n, "is not acceptable.")
      b = False

    except ValueError:
       print("Invalid integer. Try again.")
print(n,"is acceptable.")
```

outer loop continues until value input is in range

else and finally

An `else` suite (placed after all exceptions) is only executed after the try suite if no exception is raised.

```
else:
    print(f"Average = {sum/count:.2f}")
```

A `finally` suite (placed after all exceptions) is executed after the try suite whether or not an exception is raised.

```
finally:
    print("Execution complete.")
```

File I/O

I/o redirection @ command line using < >

How have we handled I/O previously?

Ideally- we want to read from a file AND allow input from the user. Likewise, we also want to BOTH output to the standard output AND to a results file during a single execution.

Now we will use code with file I/O to:

1) Open, read from, and close **input** files

2) Open, write to/append, and close **output** files

To include file I/O in our code, we must create a **file object**- a way for the program to work with that file.

Opening and Closing Files

The `open` function creates a file object and associates it with a file using a general format:

```
file_object = open(filename, mode="r", buffering=-1, encoding=None,
   errors=None, newline=None, closed=True, opener=None)
```

The **mode** string specifys how the file will be opened:

Character	Meaning
`'r'`	open for reading (default)
`'w'`	open for writing, truncating the file first
`'x'`	open for exclusive creation, failing if the file already exists
`'a'`	open for writing, appending to the end of file if it exists
`'b'`	binary mode
`'t'`	text mode (default)
`'+'`	open for updating (reading and writing)

https://docs.python.org/3/library/functions.html#open

Default: read `("r")` and text `("t")`

Example 3a: For example, if we want to open the `grades1.txt` input file:

```
infile = open("grades1.txt", "r")
```

or

```
infile = open("grades1.txt")
```

The data file will be opened and the variable `infile` will reference a file object that we can use to read data from the file.

Example 3b: To allow your user to specify the input file, first prompt them for the input file name and save their input as a string:

```
filename = input("Enter input filename: ")
```

then open the file, using the string variable rather than a string literal with the exact filename:

```
infile = open(filename, "r")
```

To open an output file to write output or results:

```
outfile = open(filename, "w")
```

CAUTION: This will CREATE the file, over-writing any existing files of that name!

Any file that is opened should be closed when finished with it. The `close` function disconnects the file from the program. Always a good practice to make sure that any data left in the buffer gets written to the file.

```
outfile.close()
```

What if an exception occurs before closing the file?

Example 3c: **Best practice**:

```
with open(filename, "r") as infile:
    ...
```

After opening the file for in read (or write) mode, it will **automatically close** it.

Writing to a Text File

Suppose we have a data file called `CSprofs.txt` consisting of strings:

```
Titus
Heckman
Sturgill
Jennings
Battestilli
```

That data file really contains:

'Titus\n Heckman\n . . . Battestilli\n'

↑ best practice

Example 4a: To create the above file and write to it:

```
with open("CSprofs.txt", "w") as outfile:
    outfile.write("Titus\n")
```

Example 4b: To append to an existing file without rewriting over it:

```
with open("CSprofs.txt", "a") as outfile:
   outfile.write("Heckman\n")
   outfile.write("Sturgill\n")
   outfile.write("Jennings\n")
   outfile.write("Battestilli\n")
```

Note: the `write` method will always write a **string** format to the output file.

Example 4c: If writing a string without a newline character to your file, you must concatenate one to the string if you want it to be treated as a separate line:

```
name = "Titus"
outfile.write(name + "\n")
```

Reading from a Text File

If we start with the data file consisting of strings from before:

```
Titus
Heckman
Sturgill
Jennings
Battestilli
```

Example 5a: We can use the `read` method to read in the **entire** file at once:

```
with open("CSprofs.txt") as infile:
     all_data = infile.read()
     print(all_data)
```

This means `all_data` is a single string that contains:

```
Titus\nHeckman\nSturgill\nJennings\nBattestilli\n
```

which requires some level of processing to separate into multiple strings.

Example 5b: To read a **single** line at a time, use the `readline` method:

```
prof1 = infile.readline()
prof2 = infile.readline()
prof3 = infile.readline()
prof4 = infile.readline()
. . .
```

What does `prof1` contain? `prof2`?

'Titus\n' 'Heckman\n'

Example 5c: To use this string elsewhere in your code, it is often convenient to NOT have the newline present. Use either the `strip` or `rstrip` method to strip the newline character off the right side of the string:

```
name1 = "Gupta\n"
name1 = name.rstrip("\n")
print(name1)
Gupta
```

or rstrip() or strip('\n')

Example 5d: A more efficient way to read line by line, using the default read mode:

```
with open("CSprofs.txt") as infile:
     for line in infile:
          print(line, end = "")
```

until EOF

Example 5e: To read the lines of an entire file in as a list, use the `readlines` method:

```
with open("CSprofs.txt") as infile:
     all_data = infile.readlines()
```

['Titus\n', 'Heckman\n', ... 'Battestilli\n']

Example 5f: Or strip the newline and create the list:

```
with open("CSprofs.txt") as infile:
     profs = []
     for line in infile:
          if line != "":
               profs.append(line.strip())
```

Example 5g: For CSV files, tokenize each line (list element), using the comma as a delimiter to create sublists:

```
for line in all_data:
     tokens = line.split(",")
     for items in tokens:
          print(item)
```

We will see more CSV options with the csv library and Pandas.

You can also strip the newline using the `replace` method:

```
line = line.replace("\n", "")
```

A Common Exception

What if your program uses file I/O and asks the user for the filename? What can go wrong?

Example 6: If the file specified is not found, the traceback error will be an `IOError` type.
Let's use a try/except block:

```
filename = input('Enter a filename: ')

try:
     with open(filename) as infile:
         contents = infile.read()
         print(contents)

except IOError:
    print('An error occurred trying to read the file', filename)
```

Best Practices

1) When you plan your exceptions, order your except clauses from most specific to most general. Consider the following hierarchy of exception classes:

```
Exception
   OSError
        FileExistsError
        FileNotFoundError
   ValueError
```

2) If you use a `with` statement to open your file, Python will clean up the system resources and close the file. If you don't use a `with`, then you should add a `finally` clause to close the file, etc.

```
   try:
        file = open(filename, "r")
        name = file.readlines()
        . . .
   except:
        . . .

   finally:
        file.close()
```